C++ & GPU Concept Primer

Oct 26, 2026 · @Jesus Leobardo Nagao

Companion to the *C++ & GPU Computing Roadmap*. Each phase introduces the core concepts in plain terms, then lists **Check your research**: statements that should match what you find. If a source contradicts one, dig deeper; either the source is outdated or this primer is oversimplifying.

Phase 0 — Setup and Monte Carlo

Compilation model. C++ is compiled ahead of time: the compiler turns each `.cpp` file (a *translation unit*) into an object file, and the linker combines them into an executable. Headers (`.hpp`) are textually pasted in by `#include` ; they declare things, while `.cpp` files define them.

CMake. CMake does not compile anything. It is a *build-system generator*: it reads `CMakeLists.txt` and writes Makefiles, Ninja files or Visual Studio projects, which then call the compiler. Modern CMake is *target-based*: you create targets (`add_executable` , `add_library`) and attach properties to them (`target_compile_features` , `target_link_libraries`).

Pseudo-random numbers. A generator like `std::mt19937_64` produces a deterministic sequence from a seed; a *distribution* (`uniform_real_distribution`) shapes those bits into the values you want. Same seed → same sequence, which is what makes experiments reproducible. `std::random_device` is for seeding, not for bulk generation.

Monte Carlo convergence. Each sample is a Bernoulli trial (inside the circle or not), so the estimate is a sample mean. By the central limit theorem its standard error scales as $\sigma/\sqrt{N}$: 100× more samples buys only 10× more accuracy. This $1/\sqrt{N}$ law is independent of dimension, which is why Monte Carlo wins for high-dimensional integrals (Phase 5).

Timing and optimization levels. Use a monotonic clock (`steady_clock`) for durations; `system_clock` can jump. Debug builds (`-O0`) keep code literal for debuggers; Release (`-O2` / `-O3`) lets the compiler inline, vectorize and reorder. Never benchmark a Debug build.

Check your research

- CMake generates build files; the compiler (clang, MSVC, nvcc) does the compiling.
- `rand()` has implementation-defined, often poor quality and a small range; `<random>` engines are the modern replacement.
- Monte Carlo error $\propto 1/\sqrt{N}$, so a log-log plot of error vs N has slope $-1/2$.
- A benchmark result is only meaningful in an optimized build and after a warm-up run.

Phase 1 — Modern C++ and fast CPU code

Templates. A template is a recipe the compiler stamps out per type: `Vector<double>` and `Vector<float>` become two separate, fully optimized classes (*instantiation*). That is why template code usually lives in headers: the compiler must see the full definition to instantiate it.

Value semantics, copies and moves. C++ objects behave like values: assigning copies them. A *move* transfers ownership of internal resources (e.g. the heap buffer inside a `std::vector`) instead of duplicating them, leaving the source valid but empty. The *rule of zero*: if your class only holds well-behaved members (`std::vector`, smart pointers), write no copy/move/destructor at all and the compiler-generated ones are correct.

RAII. Resources (memory, files, GPU buffers, locks) are acquired in a constructor and released in the destructor, so cleanup happens automatically when an object leaves scope, even on exceptions. You will reuse this to wrap CUDA and Metal buffers.

The memory hierarchy. RAM is ~100× slower than a register. CPUs hide this with caches (L1/L2/L3) that load 64-byte *cache lines*. Code that walks memory sequentially reuses each line; code that jumps around (striding down a column of a row-major matrix) wastes most of it. This is why `ikj` beats `ijk` in matmul: the inner loop becomes contiguous.

Threads and data races. A data race is two threads touching the same memory at the same time, with at least one writing and no synchronization. It is undefined behavior, not just a wrong answer. The safe pattern in N-body: each thread *writes* only its own range of bodies and only *reads* shared positions.

OpenMP. A compiler extension: `#pragma omp parallel for` splits a loop across a thread pool for you. Same parallelism as manual `std::thread`, far less code.

AoS vs SoA. Array of Structs (`Body bodies[n]`) stores x, y, z, mass together per body; Struct of Arrays stores all x's contiguously, then all y's. SoA lets SIMD units and GPUs load many consecutive values in one go.

Symplectic integrators. Leapfrog/velocity Verlet preserve the geometric structure of Hamiltonian systems, so energy error stays bounded and oscillates instead of drifting. Explicit Euler adds energy every step, so orbits spiral outward.

Check your research

- Templates are resolved at compile time; there is no runtime cost for genericity.
- `std::move` does not move anything by itself; it is a cast that *allows* a move constructor to be chosen.
- Row-major means element `(i, j)` lives at `i*cols + j`; iterating `j` in the inner loop is contiguous.

- A data race is undefined behavior even if the result looks correct in testing.
- Leapfrog is second-order accurate and symplectic; explicit Euler is first-order and neither.

Phase 2 — GPU compute

The execution model. A CPU has a few powerful cores optimized for latency; a GPU has thousands of simple lanes optimized for throughput. You write one function (a *kernel*) and launch it over a *grid* of *blocks*, each block holding up to 1024 *threads*. Each thread uses `blockIdx`, `blockDim` and `threadIdx` to compute which element it owns.

Warps and SIMT. Hardware executes threads in groups of 32 called *warps*, all running the same instruction in lockstep (SIMT). If threads in a warp take different branches (*divergence*), the branches run one after another, wasting lanes.

Memory hierarchy. From fast and small to slow and large: registers (per thread) → shared memory (per block, programmer-managed, ~100 KB per SM) → L2 cache → global memory (GBs of VRAM). Host↔device copies over PCIe are slower still, so you copy once and keep data on the GPU.

Coalescing. When the 32 threads of a warp read 32 consecutive addresses, the hardware merges them into a few wide transactions. Scattered addresses turn into many separate transactions. This is the GPU version of the cache-line lesson from Phase 1, and why SoA matters.

Occupancy and latency hiding. A GPU hides memory latency by switching to another ready warp while one waits. *Occupancy* is how many warps are resident per SM relative to the maximum; it is limited by registers and shared memory per thread. High occupancy helps, but is a means, not the goal.

Reductions. Summing N values in parallel takes $\log_2 N$ steps in a tree: each step halves the active threads. Inside a block, partial sums meet in shared memory with `__syncthreads()` between steps; inside a warp, `__shfl_down_sync` exchanges registers directly with no shared memory. The per-block results are then combined (a second kernel or `atomicAdd`).

Tiling. Naive matmul reads each input element from global memory n times. Tiling loads a block-sized tile into shared memory once and reuses it by every thread in the block, raising *arithmetic intensity* (FLOPs per byte moved).

Roofline thinking. Every kernel is limited either by memory bandwidth or by compute. SAXPY does 2 FLOPs per 12 bytes moved: hopelessly memory-bound, so judge it in GB/s against peak bandwidth. Tiled matmul reuses data heavily and can approach the compute peak, so judge it in FLOP/s.

CUDA ↔ Metal vocabulary

CUDA	Metal
kernel (`__global__`)	kernel function in a `.metal` file
thread / block / grid	thread / threadgroup / grid
warp (32)	SIMD-group (32 on Apple GPUs)
`__shared__` memory	`threadgroup` memory
`__syncthreads()`	`threadgroup_barrier(mem_flags::mem_threadgroup)`
stream	command queue / command buffer

One big difference: Apple Silicon has *unified memory*, so CPU and GPU share the same physical RAM and no PCIe copy is needed.

Stencils and stability. The heat-equation update reads each cell's 4 neighbors: lots of memory traffic, little math, so it is memory-bound. The explicit scheme is stable only if $\alpha \cdot dt/dx^2 \leqslant 1/4$ in 2D; break it and the solution oscillates and blows up.

Check your research

- Kernel launches are asynchronous: the CPU continues immediately, so time with `cudaEvent` or after `cudaDeviceSynchronize()`.
- `__syncthreads()` synchronizes threads *within one block only*; there is no barrier across blocks inside a kernel.
- Shared memory is per block and disappears when the block finishes.
- Floating-point addition is not associative, so a GPU sum may differ from the CPU sum in the last digits; that is expected, not a bug.
- On NVIDIA, warp size is 32; on Apple GPUs, the SIMD-group size is also 32.

Phase 3 — Graphics and Stable Fluids

The rendering pipeline. Vertices go through a *vertex shader* (positions them on screen), get assembled into triangles, are *rasterized* into fragments (candidate pixels), and each fragment runs a *fragment shader* that decides its color. To display a simulation you only need one trick: draw two triangles covering the screen and let the fragment shader read your field from a texture.

WebGPU object model. *Instance* → *adapter* (a physical GPU) → *device* (your logical handle) → *queue* (where work is submitted). GPU memory lives in *buffers* and *textures*. A *pipeline* bundles shaders plus fixed state; a *bind group* tells a pipeline which buffers and textures to use. Work is recorded into a *command encoder* and submitted as a command buffer. This is the same model as Metal and Vulkan, just friendlier.

WGSL. WebGPU's shading language. Entry points are tagged `@vertex`, `@fragment` or `@compute`; compute shaders declare `@workgroup_size(x, y)` and read `global_invocation_id`, the equivalent of CUDA's global thread index.

Render vs compute. A render pipeline turns geometry into pixels; a compute pipeline runs arbitrary parallel code over buffers, with no geometry involved. Your simulation is compute; the visualization is render; both share the same buffers on the same device.

Stable Fluids, step by step. Each frame applies operator splitting to the incompressible Navier-Stokes equations:

1. **Add forces:** inject velocity and dye from the mouse.
2. **Advect:** move quantities along the velocity field. *Semi-Lagrangian* advection traces each cell backward by $u \cdot dt$ and interpolates the value found there. Because it only ever interpolates existing values, it cannot create new extremes, so it is stable for any dt.
3. **Diffuse (optional):** viscosity, solved implicitly.
4. **Project:** make the velocity divergence-free. Solve the Poisson equation $\nabla^2 p = \nabla \cdot u$ for pressure (Jacobi iterations on the GPU), then subtract ∇p from u. This is the Helmholtz decomposition, and the same Chorin projection your Julia solver uses.

Vorticity confinement. Semi-Lagrangian advection is numerically diffusive and smears small vortices. Vorticity confinement adds a small force that re-amplifies existing swirls; it looks right but is not physically derived.

Check your research

- WebGPU on macOS runs on Metal; on Windows it runs on Direct3D 12 or Vulkan.
- WGSL storage buffers used in compute can be read by a later render pass in the same frame.
- Semi-Lagrangian advection is unconditionally stable but only first-order accurate and numerically dissipative.
- Jacobi converges slowly: dozens of iterations per frame is normal; multigrid or conjugate gradient converge faster.
- Stable Fluids prioritizes visual stability over accuracy; it is not a validated CFD method.

Phase 4 — Lattice Boltzmann aerodynamics

The idea. Instead of solving Navier-Stokes for velocity and pressure directly, LBM tracks *particle distribution functions* $f_i(x, t)$: the density of fictitious particles at node x moving with discrete velocity c_i. Macroscopic quantities are moments: density $\rho = \Sigma f_i$, momentum $\rho u = \Sigma f_i c_i$. A Chapman-Enskog expansion shows these moments obey the weakly compressible Navier-Stokes equations.

D2Q9. 2 dimensions, 9 velocities: rest, 4 axis neighbors, 4 diagonals, with weights 4/9, 1/9 and 1/36. The lattice speed of sound is $c_s^2 = 1/3$ (lattice units).

Collide and stream. Each time step has two phases. *Collision* (local, per node): relax f_i toward the equilibrium $f_i^{eq}(\rho, u)$ at rate $1/\tau$ (the BGK operator). *Streaming* (neighbor copy): move each f_i one node along c_i. Collision is pure arithmetic and streaming is pure memory movement, and every node is independent: a near-perfect fit for GPUs.

Viscosity and units. Kinematic viscosity is $\nu = c_s^2(\tau - 1/2) = (\tau - 1/2)/3$ in lattice units. τ must stay above 1/2; as it approaches 1/2 viscosity vanishes and BGK goes unstable. The Reynolds number $Re = U \cdot L/\nu$ is what connects your lattice to a physical flow: pick L in nodes, U in lattice units, then derive τ.

Low Mach requirement. LBM is weakly compressible with errors of order Ma^2. Keep the lattice velocity small ($U \lesssim 0.1$, i.e. $Ma \lesssim 0.17$) to approximate incompressible flow. Higher Re therefore needs more nodes, not a bigger U.

Boundary conditions. *Bounce-back*: a population hitting a solid node is reflected back where it came from, giving no-slip. In the *half-way* variant, the wall sits halfway between the fluid and solid nodes and the scheme is second-order accurate. Inlets and outlets set unknown populations from a prescribed velocity or pressure (Zou-He is the standard choice).

Forces: momentum exchange. At every bounce-back link, the population reverses direction, transferring momentum $2f_i c_i$ to the body. Summing over all boundary links each step gives the force; normalize it to $Cd = F_x / (\frac{1}{2}\rho U^2 L)$ and $Cl = F_y / (\frac{1}{2}\rho U^2 L)$.

Validation numbers. Cylinder in a free stream: steady symmetric wake for $Re \lesssim 47$; periodic von Kármán shedding above, with Strouhal $St = fD/U \approx 0.16$–0.17 at Re = 100. Channel walls (blockage) shift both Cd and St upward, so compare against data with similar blockage.

MLUPS. Million Lattice Updates Per Second = nodes × steps / (seconds × 10^6). The standard LBM performance metric; on a GPU it is bounded by memory bandwidth, since each node reads and writes 9 values per step.

Check your research

- LBM recovers incompressible Navier-Stokes only in the low-Mach, low-Knudsen limit.

- $\nu = (\tau - 1/2)/3$ in lattice units; $\tau \to 1/2$ means instability.
- Half-way bounce-back is second-order accurate; full-way (on-node) bounce-back is first-order.
- A staircase (voxelized) airfoil surface adds error at the leading edge; curved-boundary schemes (e.g. Bouzidi) reduce it.
- 2D LBM at Re ~10^3 will not match high-Re wind-tunnel airfoil data (Re ~10^6); compare at matched Re.

Phase 5 — Derivatives pricing

Options. A European call pays $\max(S_T - K, 0)$ at maturity T; a put pays $\max(K - S_T, 0)$. S is the underlying price, K the strike. The question is what that future payoff is worth today.

Risk-neutral pricing. Because an option can be replicated by continuously trading the underlying and a risk-free bond, its price cannot depend on investors' risk preferences. So you may pretend everyone is risk-neutral: the underlying drifts at the risk-free rate r, and the price is the discounted expected payoff, $V = e^{(-rT)} \cdot E_Q[\text{payoff}]$. The real-world expected return μ does not appear.

Geometric Brownian motion. Under Q, $dS = rS\,dt + \sigma S\,dW$. Its exact solution, $S_T = S_0 \cdot \exp((r - \sigma^2/2)T + \sigma\sqrt{T} \cdot Z)$ with $Z \sim N(0, 1)$, lets you jump straight to maturity with no time-stepping error. The $-\sigma^2/2$ term is the Itô correction: without it, the expected price would drift too high.

Black-Scholes. For European options under GBM, the expectation has a closed form: $C = S_0 N(d_1) - Ke^{(-rT)} N(d_2)$, with $d_1 = [\ln(S_0/K) + (r + \sigma^2/2)T] / (\sigma\sqrt{T})$ and $d_2 = d_1 - \sigma\sqrt{T}$. This is your ground truth for validating every Monte Carlo variant.

Put-call parity. $C - P = S_0 - Ke^{(-rT)}$, for any model without arbitrage. It makes a cheap, model-independent unit test.

The Greeks. Sensitivities of the price: delta $\partial V/\partial S$ (hedge ratio), gamma $\partial^2 V/\partial S^2$ (how fast delta changes), vega $\partial V/\partial \sigma$, theta $\partial V/\partial t$, rho $\partial V/\partial r$. Finite-difference checks must use the *same random numbers* for the bumped and base runs (common random numbers), or the noise swamps the derivative.

Monte Carlo pricing. Simulate N terminal prices, average the payoffs, discount. The standard error is $s/\sqrt{N}$, where s is the sample standard deviation of the discounted payoffs; report price ± 1.96·SE as a 95 % interval.

Variance reduction. *Antithetic variates*: for each Z also use −Z; the two payoffs are negatively correlated, so their average has lower variance. *Control variates*: simulate a related quantity Y with a known expectation (the discounted underlying, or a closed-form

option) and correct the estimate by $\beta(\bar{Y} - E[Y])$. Both cut the error at the same N, which beats brute-forcing more paths.

Path-dependent options. Asian options pay on the *average* price along the path; barrier options activate or die if the price crosses a level. These need time-stepping and usually have no closed form, which is exactly where Monte Carlo earns its keep. Monitoring a barrier only at discrete steps misses crossings between steps, which biases the price (Broadie-Glasserman-Kou gives a correction).

GPU random numbers. Each thread needs an *independent* stream. cuRAND's `curand_init(seed, subsequence, offset, &state)` gives each thread its own subsequence of the same generator, which guarantees no overlap. Seeding every thread with `seed + threadId` on a separate generator does not.

Check your research

- Under the risk-neutral measure the drift is r, not the stock's expected return μ.
- Black-Scholes assumes constant σ and r, no dividends (in the basic form), continuous trading and lognormal prices.
- MC standard error $\propto 1/\sqrt{N}$ regardless of the number of time steps or assets.
- The geometric-average Asian option has a closed form; the arithmetic-average one does not.
- Discrete barrier monitoring overprices knock-out options relative to continuous monitoring.

Phase 6 — Trading systems

Limit order book. The list of resting buy orders (*bids*) and sell orders (*asks*) at each price. The highest bid and lowest ask are the *best bid/offer*; the gap between them is the *spread*. A *limit order* waits in the book at its price; a *market order* takes liquidity immediately at the best available prices.

Price-time priority. When an incoming order crosses the spread, it matches against the best price first, and among orders at the same price, the oldest first (FIFO). A large order can *walk the book*, filling across several price levels, and partially fill, leaving the remainder resting.

Why integer prices. Prices move in fixed *ticks* (e.g. $0.01). Storing them as integers (number of ticks) makes equality and ordering exact; `double` cannot represent 0.1 exactly, so two "equal" prices may compare unequal and land on different levels.

Data structure trade-offs. A balanced tree (`std::map`) gives O(log n) insertion and ordered traversal. Production engines often use an array indexed by tick offset from a reference price, giving O(1) level access and cache-friendly memory, at the cost of a

bounded price range. Cancels need an id → order lookup (hash map) to avoid searching.

Latency percentiles. Mean latency hides the outliers that matter. p50 is the median; p99 is the value 99 % of operations beat. Trading systems care about the tail (p99, p99.9), driven by allocations, cache misses and OS jitter.

Event-driven backtesting. Replays history as a time-ordered stream of events: market data → strategy emits signals → portfolio turns signals into orders → simulated broker returns fills. The strategy sees only events up to the current time, which mirrors live trading and makes look-ahead bias structurally harder (unlike vectorized backtests over whole arrays).

The biases that kill backtests.

- *Look-ahead bias*: using information not yet available at decision time (e.g. today's close to trade at today's open).
- *Survivorship bias*: testing only on stocks that still exist today, which excludes the ones that went bankrupt.
- *Overfitting / data snooping*: trying many parameter sets and reporting the best one. With enough trials, something always looks good by chance.
- *Ignoring costs*: commissions, spread and slippage can erase a high-turnover strategy entirely.

Performance metrics. *Sharpe ratio*: mean excess return / standard deviation of returns, annualized (× $\sqrt{252}$ for daily data). *Max drawdown*: the largest peak-to-trough drop in equity. *Walk-forward validation*: fit on a window, test on the next unseen window, roll forward, and only report the out-of-sample results.

Check your research

- Orders at the same price are filled first-in, first-out under price-time priority.
- Daily Sharpe annualizes by $\sqrt{252}$, not 252.
- `yfinance` data is free but survivorship-biased and adjusted retroactively for splits and dividends; fine for learning, not for publishable research.
- A strategy that only works with zero transaction costs does not work.
- The more strategies you test, the higher the bar for any single Sharpe ratio to be meaningful (López de Prado's deflated Sharpe ratio formalizes this).

Data science track

Bias-variance trade-off. Expected test error = bias² + variance + irreducible noise. Simple models underfit (high bias); flexible models overfit (high variance). Every technique below is a way to sit at the right point on that curve.

Train/test and cross-validation. Training error always shrinks with complexity, so it says nothing about generalization. *k-fold cross-validation* splits data into k parts, trains on k−1 and tests on the held-out one, rotating; the average estimates test error. Any preprocessing (scaling, imputation, feature selection) must be fit *inside* each fold, or information leaks.

Regularization. Add a penalty on coefficient size to the loss. *Ridge* (L2) shrinks all coefficients smoothly; *Lasso* (L1) drives some exactly to zero, doing feature selection. The penalty strength λ is chosen by cross-validation. Standardize features first, or the penalty treats them unequally.

Trees and ensembles. A decision tree splits the feature space into boxes; alone it overfits. *Random forests* average many trees trained on bootstrap samples with random feature subsets (reduces variance). *Gradient boosting* adds small trees sequentially, each fitting the previous ensemble's errors (reduces bias). Boosting usually wins on tabular data but needs more tuning.

Feature importance. Impurity-based importance from trees is biased toward high-cardinality features; *permutation importance* on held-out data is more trustworthy. Neither implies causation.

Surrogate models. A cheap statistical model trained on outputs of an expensive simulator, here predicting Cl/Cd from (α, Re) without running LBM. Use it to interpolate within the sampled range, never to extrapolate beyond it; sample inputs with a space-filling design (e.g. Latin hypercube) rather than a coarse grid.

Time series validation. Financial data is ordered in time and serially correlated, so random k-fold leaks the future into training. Use forward-chaining splits. López de Prado's *purged* CV removes training samples whose label windows overlap the test period, and adds an *embargo* gap after it.

Check your research

- Cross-validation estimates generalization error; the final model is then refit on all training data.
- Lasso can zero out coefficients; Ridge cannot.
- Random forests mainly reduce variance; boosting mainly reduces bias.
- Never use random shuffling to split time-series data for evaluation.
- A model with great accuracy on a leaked feature (one known only after the outcome) is worthless in production.